



CS1004 – Object Oriented Programming

Lecture # 15
Monday, April 11, 2022
Spring 2022
FAST – NUCES, Faisalabad Campus

Muhammad Yousaf



2

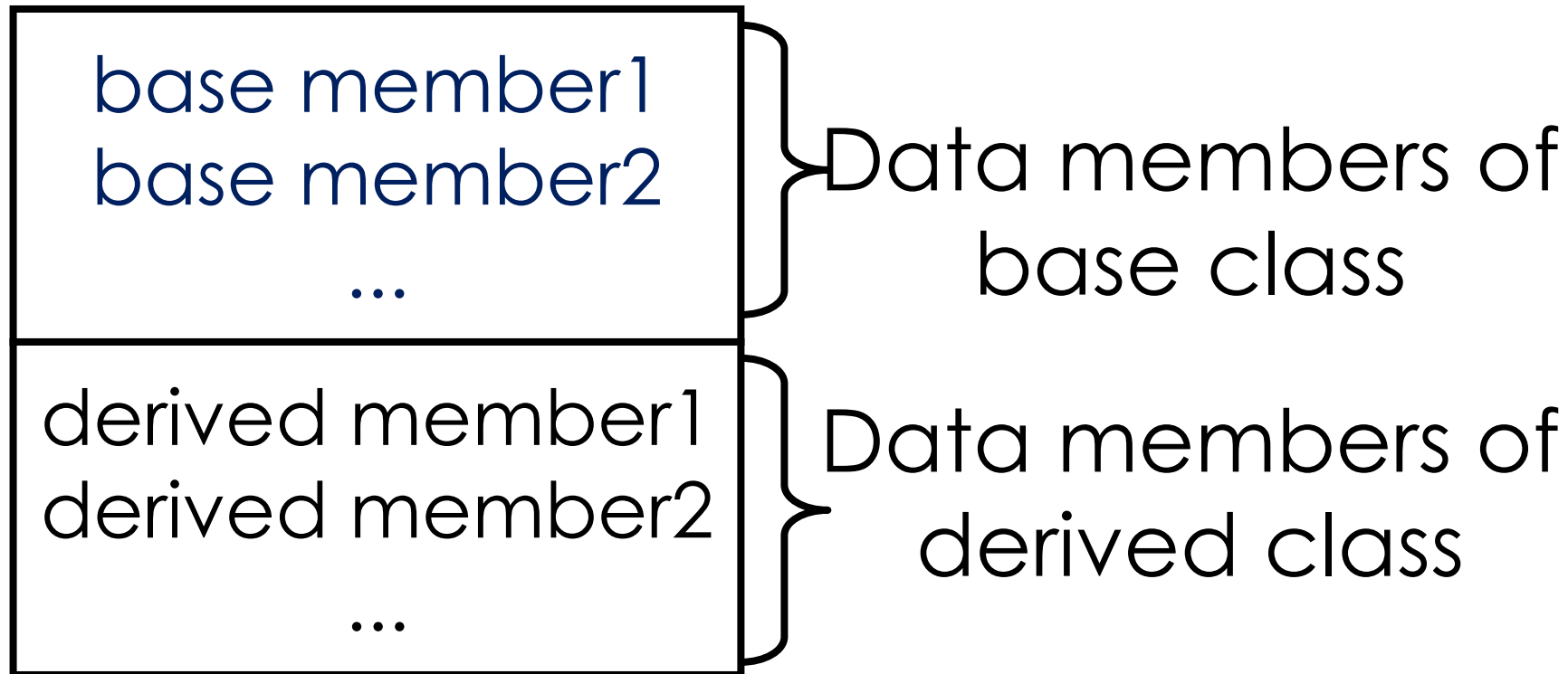
Inheritance

Task

- Write a class employee that contains attributes of employee id and scale. The class contains member functions to input and output these attributes. Write a child class Manager that inherits Employee class. The child class has attributes of manager name and his department and member function to show all attributes(employee id, scale, name, department)

Memory footprint

- The object of derived class is represented in memory as follows



Memory footprint

- ➡ Every object of derived class has an anonymous object of base class

Constructors

- The anonymous object of base class must be initialized using constructor of base class
- When a derived class object is created the constructor of base class is executed before the constructor of derived class

Constructors

base member1
base member2
...

Base class constructor
initializes the anonymous
object

derived member1
derived member2
...

Derived class constructor
initializes the derived class
object

Private Inheritance

- If the user does not specify the type of inheritance, then the default type is private inheritance

```
class Derived : private Base { ... };
```

- Is equivalent to

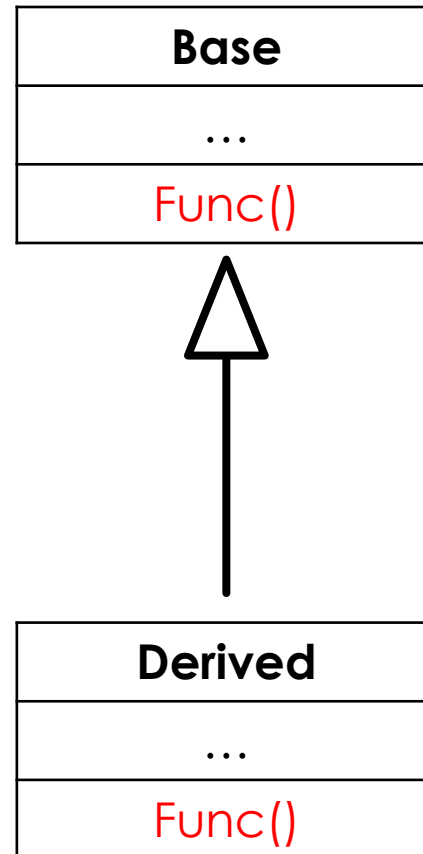
```
class Derived : Base { ... };
```

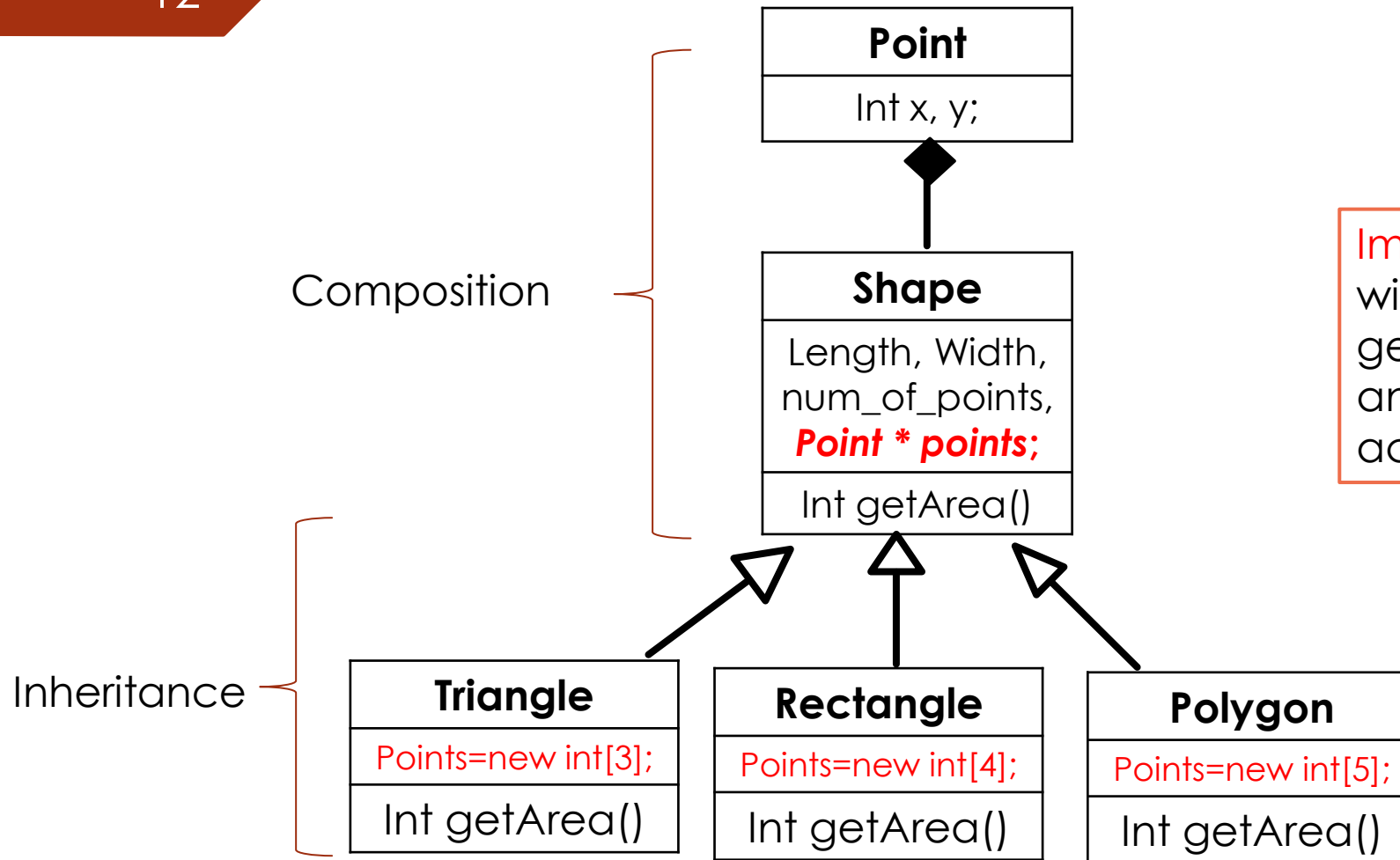

Function Overriding

Overriding member functions of base class

- ➡ Derived class can override the member functions of its base class
- ➡ To override a function the derived class simply provides a function with the same signature as that of its base class

Overriding





Implement this class hierarchy with all the necessary setters, getters, constructors, destructors and display functions accordingly!

Overriding a function of **Person** in **Student**

```
class Person {  
    char *name;  
    int age;  
public:  
    void getAge();  
    void Print();  
};  
class Student:public Person {  
    int rollNo;  
public:  
    void Print();  
};
```

Overloading vs. Overriding

- ➡ **Overloading** is done within the scope of one class
- ➡ **Overriding** is done in scope of parent and child
- ➡ **Overriding** within the scope of single class is error due to duplicate declaration

Overriding within a class

```
class Student{  
    int rollNo;  
public:  
    void Print();  
    void Print(); //Compilation Error  
};
```

Access a method

```
class Student
{
protected:
    int age;
    int rollNo;
    char *name;
public:
    void Set(int age, int rollNo,
            char* name);
};
```

```
int main()
{
    Student st1;
    UnderGradStudent gst1;
    st1.Set(26, 1024, "TestStudent"); //Calls
    //Set from base class
    gst1.Set(28, 2048, "Student", "Supervisor"); //Calls
    //Set from derived class
}
```

```
class UnderGradStudent :
public Student
{
private:
    char *ProjSupervisor;
public:
    void Set(int age, int rollNo, char* name,
            char* SName)
    {
        Set(age, rollNo, name); //Calls Set
                                //function for Student
        ProjSupervisor = new char[strlen(SName)+1];
        strcpy(ProjSupervisor, SName);
    }
};
```


Overriding member function of base class

- ➡ Derive class can override member function of base class such that the working of function is totally changed

```
class Person {  
public:  
    void Walk();  
};  
class ParalyzedPerson : public Person {  
public:  
    void Walk();  
};
```

Overriding member function of base class

- ➡ Derive class can override member function of base class such that the working of function is similar to former implementation

Overriding member function of base class

```
void Student::Print()
{
    cout << "\nName = " << name;
    cout << "\nAge = " << age;
    cout << "\nRoll No = " << rollNo;
}
```

```
void UnderGradStudent::Print()
{
    cout << "\nName = " << name;
    cout << "\nAge = " << age;
    cout << "\nRoll No = " << rollNo;
    cout << "\nFYP Supervisor = "
        << ProjSupervisor;
}
```

Overriding member function of base class

```
void Student::Print()
{
    cout << "\nName = " << name;
    cout << "\nAge = " << age;
    cout << "\nRoll No = " << rollNo;
}
```

```
void UnderGradStudent::Print()
{
    Print();
    cout << "\nFYP Supervisor = "
        << ProjSupervisor;
}
```

What will be the output if the Print function of UnderGradStudent class is called?

No output as it will recursively call Print of UnderGradStudent class

Overriding member function of base class

```
void Student::Print()
{
    cout << "\nName = " << name;
    cout << "\nAge = " << age;
    cout << "\nRoll No = " << rollNo;
}
```

```
void UnderGradStudent::Print()
{
    Student::Print();
    cout << "\nFYP Supervisor = "
        << ProjSupervisor;
}
```

Now if the Print of Derived class is called it will call the Print of Base class Student.

Overriding and pointers

- The pointers must be used with care when working with overridden functions

```
int main()
{
    UnderGradStudent ugSt1;
    ugSt1.Set(26, 1024, "TestName", "TestSupervisor");
    UnderGradStudent *ptrUG = &ugSt1;
    ptrUG->Print();

    Student * ptrSt = &ugSt1;
    ptrSt->Print();

    return 0;
}
```

Output:

Name = TestName
Age = 26
Roll No = 1024
FYP Supervisor = TestSupervisor

Name = TestName
Age = 26
Roll No = 1024

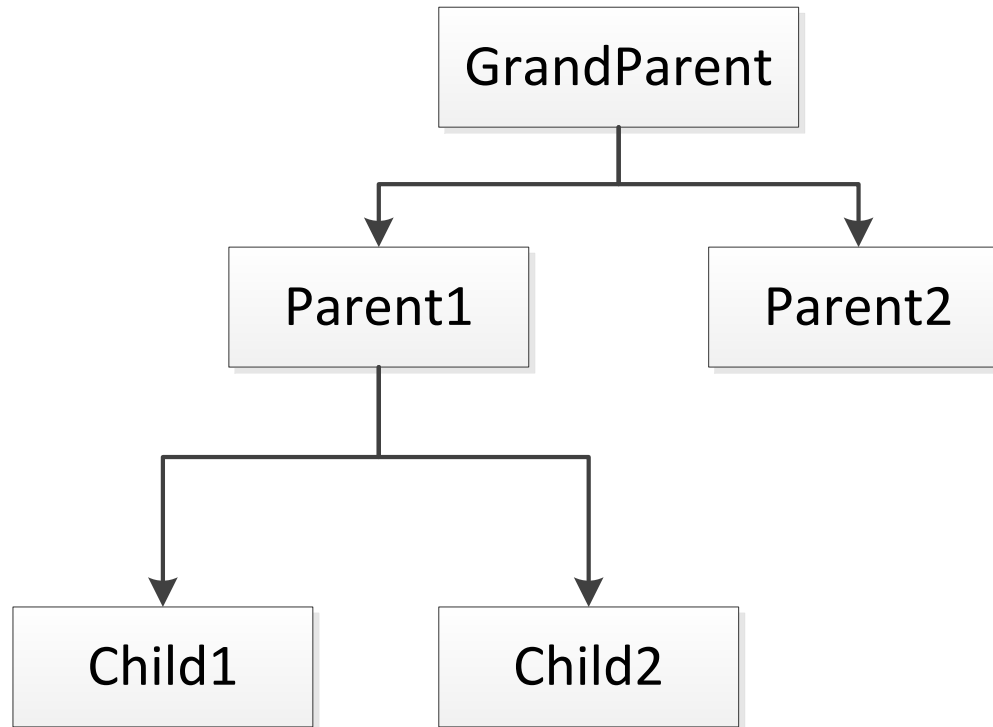
Overriding and pointers

- ➡ The pointers must be used with care when working with overridden functions
- ➡ The pointer of parent class point to an object of the child class will call the function of the base class.

Hierarchy of inheritance

- ➡ We represent the classes involved in inheritance relation in tree like hierarchy

Hierarchy of inheritance



Direct Base class

- ➡ A direct base class is explicitly listed in a derived class's header with a colon (:)

```
class Child1 :public Parent1
```

```
...
```

Indirect Base class

- An indirect base class is not explicitly listed in a derived class's header with a colon (:)
- It is inherited from two or more levels up the hierarchy of inheritance

```
class GrandParent {};  
class Parent1 : public GrandParent {};  
class Child1 : public Parent1 {};
```

Base initialization

- ➡ The child can only perform the **initialization of direct base** class through **base class initialization list**
- ➡ The child **can not perform** the initialization of an **indirect base class** through **base class initialization list**

Base initialization: Example

```
class GrandParent {
    int gpData;
public:
    GrandParent() : gpData(0) { ... }
    GrandParent(int i) : gpData(i) { ... }
    void Print() const;
};

class Parent1 : public GrandParent {
    int pData;
public:
    Parent1() : GrandParent(),
    pData(0) {
        ...
    }
};
```

```
class Child1 : public Parent1 {
public:
    Child1() : Parent1() { ... }
    Child1(int i) : GrandParent(i) //Error
    {
        ...
    }
    void Print() const;
};
```

Overriding

- Child class can override the function of GrandParent class

```
void GrandParent::Print() {  
    cout << "GrandParent::Print"  
        << endl;  
}  
void Child1::Print() {  
    cout << "Child1::Print" << endl;  
}
```

```
int main() {  
    Child1 obj;  
    obj.Print();  
    obj.Parent::Print();  
    obj.GrandParent::Print();  
    return 0;  
}
```

Output is as follows

```
Child1::Print  
GrandParent::Print  
GrandParent::Print
```

Questions

